COMP2001: ARTIFICIAL INTELLIGENCE METHODS – LABS 8 & 9 EXERCISES

RECOMMENDED TIMESCALES

This lab exercise is designed to take no longer four hours, and it is recommended that you start and complete this exercise within the two weeks so that you have the time to ask related questions to the project during the remaining computing sessions. The contents of this exercise were covered in lecture 7.

GETTING SUPPORT

The computing exercises have been designed to facilitate flexible studying and can be worked through in your own time or during timetabled lab hours. It is recommended that you attend the labs, even if you have completed the exercises in your own time, to discuss your solutions and understanding with one of the lab support team and/or your peers. It is entirely possible that you might make a mistake in your solution which leads to a false observation and understanding.

OBJECTIVES

The purpose of this lab exercise is to gain experience with using the HyFlex Framework to implement a reinforcement learning-based hyper-heuristic which selects pairs of mutation and local search heuristics to form an iterated local search hyper-heuristic approach for solving the cross-domain search problem.

- HyFlex Introduction.
- Creating a project with HyFlex.
- Implementation of a reinforcement learning-based selection hyper-heuristic with analysis of heuristic selection of mutation-local search heuristic pairings.

LEARNING OUTCOMES

By completing this lab exercise, you should meet the following learning outcomes:

- Students should gain experience using the HyFlex Framework API.
- Students should understand, and be able to implement, the components of selection hyper-heuristics.
- Students should be able to critically evaluate learning-based heuristic selection methods in a selection hyper-heuristic context.

TASK 1 – HYFLEX INTRODUCTION

An introduction and overview of the HyFlex framework and CHeSC 2011 competition was given in lecture 7 on hyper-heuristics part 1. It is important that you understand the concepts before attempting this exercise, hence, you should revisit that lecture if needed.

You can find an updated version of the CHeSC 2011 webpages at CHeSC: Cross-Domain Heuristic Search Competition (nott.ac.uk) which contains:

- A tutorial: https://www.cs.nott.ac.uk/~pszwj1/chesc2011/hyflex_tutorial.html
- Benchmarking tools: https://www.cs.nott.ac.uk/~pszwj1/chesc2011/benchmarking.html
- And description of the framework: https://www.cs.nott.ac.uk/~pszwj1/chesc2011/hyflex_description.html

Note that while the webpages do include download links to the framework and original benchmark tool, please **use the ones on Moodle** that have timer fixes and a more representative benchmark.

An additional HyFlex tutorial worksheet is provided on Moodle for further reading if this makes it easier to understand than the original webpages.

WHICH VERSION OF HYFLEX SHOULD I USE (THREAD TIMER VS WALL TIMER)

The original implementation of the HyFlex framework as used for the CHeSC 2011 competition uses a special API which is meant to keep track of the time that the hyper-heuristic thread is in a RUNNING state. A few modern operating systems and hardware architectures seem to be playing with what the timer is using to facilitate power saving modes causing this timer to under-count the amount of time the thread has been running. This means that when evaluating a hyper-heuristic for a fixed nominal time limit, your experiments might run for much longer than defined giving an unfair advantage (and a potential that your hyper-heuristic does not finish within the time limit of the in-lab practical exam) so it is important that you identify whether your device and configuration is affected by this.

There is a **TimerTest.jar** file in the libraries archive on Moodle which you should run from the command line via **java -jar TimerTest.jar** which will tell you if the timer significantly deviates from real time. To run this test, restart your computer, wait for everything to load, and close any programs/apps that may automatically start. Then, run the Java jar file via the command line and wait at least 30 seconds. We believe that the problem is limited to Windows 11 laptops when ran on battery power or when the power plan is set to an equivalent of power saver, but this may evolve to other configurations/operating systems/devices over time.

If the timer does not deviate significantly, you should use the **HyFlex_1.0.0_ThreadTimer.jar** which is the original CHeSC 2011/HyFlex library implementation, otherwise you should use the **HyFlex_1.0.1_WallTimer.jar** library which only uses the wall clock time (ensure you close as many background processes as possible when running timed experiments to maximise your computational power during the time limit).

TASK 2 – CREATING A PROJECT INCLUDING HYFLEX

You may choose to include the libraries and source code into the existing lab exercise project or create a new project solely for HyFlex. The latter may be better and act as a starting point for the implementation part of the project coursework.

If you have forgotten how to set up a Java project in IntelliJ and/or how to include the libraries, you should refer to the instructions from lab 1.

Included in this lab are:

- Three JAR files containing the HyFlex Framework ((HyFlex_1.0.0_ThreadTimer.jar **or** HyFlex_1.0.1_WallTimer.jar) **and** PersonnelScheduling.jar); you need to import **two** of these as libraries into your project as described in the section "Which Version of Hyflex Should I Use (Thread Timers vs Wall Timers)".
- A new test frame for HyFlex and cross-domain search and a utilities class which contains some potentially useful methods for the lab exercises and the project coursework implementation. These should go into a package **com.aim**.
- An exercise specific runner and configuration class should go in **com.aim.runners**.
- The hyper-heuristic to be implemented along with supporting classes which should go into **com.aim.hyperheuristics**.

TASK 3 – A SIMPLE REINFORCEMENT LEARNING ITERATED LOCAL SEARCH BASED SELECTION HYPER-HEURISTIC FOR CROSS-DOMAIN SEARCH [IMPLEMENTATION]

You are given 3 Classes, `RLILS_AM_HH`, `RouletteWheelSelection`, and `HeuristicPair`. You will need to complete the implementations for `RLILS_AM_HH` and `RouletteWheelSelection` to implement the Reinforcement Learning based Iterated Local Search Hyper-heuristic.

Now that we are using HyFlex, your hyper-heuristic will be called a **single** time. You must implement the while loop containing the termination condition. Execution of your solution is handled by the `Lab8ExerciseRunner` Class that is provided for you and the experimental parameters can be configured in `Lab8TestFrameConfig`.

Everything for the hyper-heuristic can be implemented within the `solve(ProblemDomain problem)` method of `RLILS_AM_HH.java` however you should implement Roulette Wheel Selection within `RouletteWheelSelection.java`. We have also specified five additional methods which we ask that you implement to perform the respective operations. This has the advantage of making RWS easier to implement and debug during the lab! These methods are as follows:

```
1. int getScore( int[] hs );
2. int getTotalScore();
3. void incrementScore( int[] hs );
4. void decrementScore( int[] hs );
5. int[] performRouletteWheelSelection();
```

The configuration which you are asked to use is as follows:

- Roulette Wheel Selection:
  - Default score = 5
  - Lower_bound = 1
  - Upper_bound = 10
- Target problems and instances:
  - MAX-SAT (3,11)
  - TSP(0,6)
  - 0-1 Bin Packing (7,11)
- Run time = 30 nominal (competition) seconds.

- **`Trials per instance = 5.`** (This is to allow you to complete the exercise within a reasonable time for the lab but may not be best practice for designing/evaluating your hyper-heuristic approach in the project).

There are also two other methods which we have supplied which you can use when debugging your code. These are `printHeuristicIds()` and `printHeuristicScores()` and will print out (into the console) the heuristic IDs and their associated scores respectively.

HEURISTIC SELECTION – ROULETTE WHEEL SELECTION (FITNESS PROPORTIONATE SELECTION)

Roulette Wheel Selection (RWS), also known as Fitness Proportionate Selection, is a reinforcement learning based heuristic selection method. The basic idea behind RWS is that each heuristic has associated with it a score which is bounded between a lower and upper bound. If the application of a heuristic results in an improving move (`f(s') < f(s)`) then the score for that heuristic is incremented such that the probability of selecting it again is increased. In contrast, if the application of a heuristic results in a worsening move (`f(s') > f(s)`), then its score is reduced such that the probability of selecting it again in decreased. The scores for each heuristic are then used directly as "proportions" or probability weights which **can be** calculated as shown in Equation (1). When it comes to selecting a heuristic using RWS, a random number is generated and used to select a heuristic. An illustration is given below of this process.

$$P(h_i) = \frac{score(h_i)}{\sum_{j=0}^{N-1} score(h_j)} \quad (1)$$

Given a set of heuristic IDs and scores:

Heuristic ID	0	1	2	3	4
Score	1	4	3	1	1
Proportion	10%	40%	30%	10%	10%

Generate a random number, `random_value`, between 0 (inclusive) and the sum of scores (exclusive).

$$random\_value \leftarrow 7$$

Then select the heuristic by choosing the first heuristic to have a cumulative score, as calculated in Equation (2), greater than the $random\_value$.

$$cumulative\_score(h_i) = \sum_{j=0}^{i-1} score(h_j) \quad (2)$$

Heuristic ID	0	1	2	3	4
Score	1	4	3	1	1
Proportion	10%	40%	30%	10%	10%
Cumulative Score	1	5	8	9	10
Selection Boundaries	[0]	[1-4]	[5-7]	[8]	[9]

Here we choose the heuristic as heuristic ID 2 because $cumulative\_score(h_2) = 8, 8 > 7$, and $cumulative\_score(h_1) = 5$, and $5 \not> 7$.

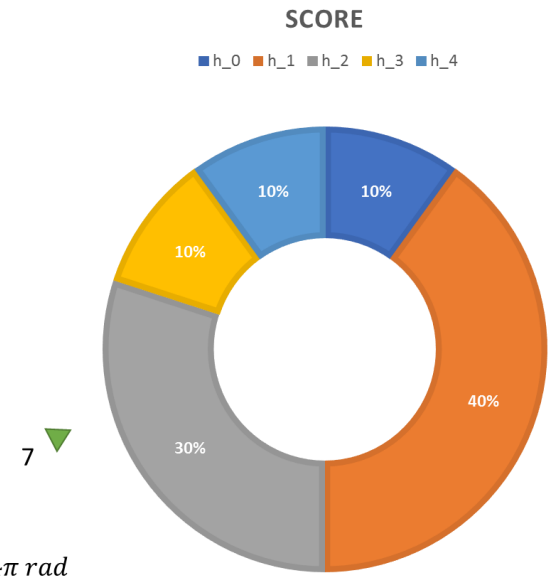
Heuristic ID	0	1	2	3	4
Score	1	8	2	1	3
Proportion	6.7%	53.3%	13.3%	6.7%	20%
Cumulative Score	1	9	11	12	15
Selection Boundaries	[0]	[1-8]	[9-10]	[11]	[12-14]

Here we choose the heuristic as heuristic ID 1 because $cumulative\_score(h_1) = 9$, 9 >1, and $cumulative\_score(h_0) = 1$, and $1 \not> 1$.

Another way to visualise this is as a roulette wheel where each heuristic has an associated segment, and its size is proportioned based on its score. The following two examples are the same as the above two but displayed using this method.

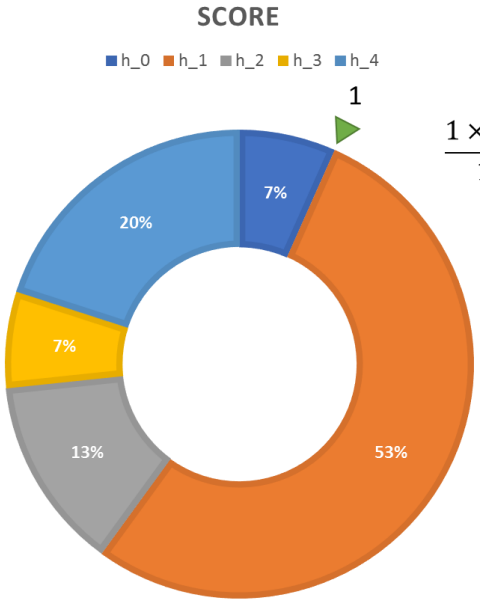
```
1 | INPUT: heuristic_ids, heuristic_scores.
2 | total_score = sum(heuristic_scores);
3 | value = random ∈ [0, total_score];
4 | WHILE cumulative_score < value DO
5 |     h <- selectNextHeuristicFromHeuristicIDs;
6 |     chosen_heuristic <- h;
7 |     cumulative_score += score(h);
8 | END_WHILE
9 | return chosen_heuristic;
```

Code 2 - Pseudocode for selection part of Roulette Wheel Selection.



7

$$\frac{7 \times 360}{10} = 252° = 1.4\pi\ rad$$



1

$$\frac{1 \times 360}{15} = 24° = 0.13\pi\ rad$$

MOVE ACCEPTANCE – ALL MOVES

For the move acceptance component, you will implement All Moves acceptance. As its name suggests, all moves are accepted and never rejected. In theory, and the idea behind such a method is that, if the heuristic selection method chooses the "best" heuristic at each iteration of the search then we should not need a move acceptance criterion that filters (rejects) candidate solutions. Hence, all moves acceptance accepts all moves. In practice, the use of move acceptance methods capable of rejecting certain moves improves the cross-domain performance of selection hyper-heuristics so you may experiment with others as seen in lectures.

TASK 4 – EXPERIMENTATION

4A – SETTING UP THE NOMINAL TIME

HyFlex uses time as its termination criterion. This means that for a fair comparison, your machine will need to be benchmarked using the **modern benchmarking tool which can be found on the Moodle page**. Do not use the benchmark tool on the CHeSC 2011 webpage. This does not work on Mac OS X/OS X/macOS and does not provide an accurate duration on modern hardware / operating systems.

You should download and run the benchmark tool to work out how long your computer should run for to achieve 10 nominal competition minutes. You should then set the `MILLISECONDS_IN_TEN_MINUTES` variable in `HyFlexTestFrame.java` to this value. The test frame(s) will use this value to figure out the run time for 60 nominal seconds. **You should do this for every computer that you use since their speeds will be different**. Moreover, the tool is not suitable for cloud computing services or even running experiments concurrently on the same machine as individual CPU core speeds can differ and other processes may interrupt the main thread(s)!

4B – COMPARISON OF REWARD, PUNISHMENT, NEUTRAL, AND RANDOMISED ACTION HEURISTIC SCORES IN A REINFORCEMENT LEARNING SELECTION HYPER-HEURISTIC FOR CROSS-DOMAIN SEARCH

Purpose of exercise: the purpose of the following exercise is to first investigate the effects of reward and punishment in a cross-domain search context for handling equal cost moves. Furthermore, you will be required to compare the performance of a selection hyper-heuristic's cross-domain performance using the CHeSC 2011 competition's F1 scoring mechanism to enable you to design and develop an effective selection hyper-heuristic in the project coursework.

The proposed reinforcement learning mechanism that you should have already implemented handles equality of solutions by rewarding the heuristic applied. In other words, if after applying the heuristic `h` to solution `s` to get `s'` and f(s') <= f(s) then h.reward() else h.punish().

The question is why should we necessarily reward equality and does handing this by "doing nothing", punishing, or randomly (rewarding, punishing, not changing) the selected heuristic outperform the original approach? For this set of exercises, you should run the `Lab8ExerciseRunner` and record your results. Then, you should modify your implementation of `RLILS_AM_HH` to (do nothing/punish/randomly adjust) heuristics that result in equality and re-run your experiments, recording your results for later analysis.

Note that when the hyper-heuristic is run, as output there will be a CSV file created in the project root called `RL-ILS_HH.csv`. This file will contain the results of the experiments including the objective values of the best solutions found for each problem domain, instance, and trial. **You should create a copy of this file before changing the algorithmic parameters so that you can compare the performance of the RL-ILS hyper-heuristic across different sets of parameters.**

Once you have ran all necessary experiments, you should collect all the data into an Excel spreadsheet and calculate the cross-domain F1 scores for all three approaches.

- Does the result surprise you?
- Does the best approach perform the best for solving all problem instances?
- What about for problem domains in general?
- **Discuss** your findings with your peers in the lab exercise Moodle forum Lab Support and Discussions (link here)

EXPECTED OUTPUT

No expected output is given since any slight variation in your implementation and your computer specification has the potential to significantly affect the output, even if your implementation is correct!

TASK 5 – FORMATIVE ASSESSMENT (MOODLE QUIZ)

There is no formative assessment for this lab as the intention is to facilitate you to gain experience with the HyFlex framework API and get you set up and ready for completing the project coursework.

If you would like **feedback** on your work (implementation/experimental design/F1 score calculation/etc…) please come along to one of the timetabled lab sessions and speak to one of the teaching team.

ADDITIONAL INFORMATION AND TIPS

A `LinkedHashMap` has been initialised in `RouletteWheelSelection.java` for you. Being "linked", at least in Java, ensures that the heuristic IDs (the key set) remain ordered in the way that they were added. The use of a Map allows for a convenient way to store the **heuristics (as keys)** and **heuristic scores (as values)**.

The Javadocs for some of the additional data structures can be found at:
https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/util/LinkedHashMap.html
In particular, you should pay attention to the methods:

- `put(Object o)`
- `get(Object o)`
- `values()`
- `keySet()`

Below are some methods which may come in useful when implementing Roulette Wheel Selection:

How do I access the heuristics within the HeuristicPair Record Class?

> See an explanation of Java Records at Java 14 Record Keyword | Baeldung (https://www.baeldung.com/java-record-keyword).

How do I get the list of heuristic IDs?

Being stored in a (Linked)HashMap, you can retrieve the heuristics as a Set of HeuristicPair. You can get this by calling the `keySet()` method on the HashMap.

How do I get the score for each heuristic ID?

Since the scores are stored as values in a Map, the score of a heuristic can be retrieved by calling the `get( HeuristicPair hs )` method on the HashMap.

How do I set the score of a heuristic?

Because we have chosen a Map data structure, you can easily update the score of a heuristic with ID `id` by calling the `put( HeuristicPair hs, Integer score )` method on the HashMap. It doesn't matter if the entry already exists in the Map since the definition of a Map is to **replace** an element if the key already exists.

I cannot iterate over the heuristic IDs using a for loop!

Being a Set of Integers, you cannot use a standard for loop to iterate over the contents. The most synonymous way that you can do this is to use a foreach loop which is written as follows:

```
for(int i = 0; i < array.length; i++) {  // standard for-loop
    Element e = array[i];
    …
}
```

Goes to:

```
for(Element e : array) {                 // foreach loop
    …
}
```

In our case, the "array" will be replaced by the `keyset` (feel free to use the `entrySet` if you know how this works!) and the type of e will be `Integer`.